

Stoquify Development Blocker Elimination Execution Prompt

Evidence-led gate isolation and blocker-elimination review brief

EXECUTION PROMPT

Prepared 2026-08-11

Evidence-bound | secret-safe | approval-gated | resumable

No readiness claim without target-observed evidence.

Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses `not applicable` with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Workspace

Work in:

```
E:\ohada saas\Focused projects\stoquify
```

Review Type

This is an evidence-led development-blocker isolation, classification, and elimination-strategy review.

Do not implement remediations during this review.

Primary Source Report

Begin with:

```
what-next/ohada-compliance-deferral-architecture-review.md
```

Corroborate its conclusions against current repository truth. Do not assume that command results, gate counts, package scripts, or blockers recorded in the report are still current.

Also inspect, where relevant:

- 1 `what-next/platform-progress-recovery-assessment-2026-08-10.md`
- 1 `docs/country-pack/COUNTRY_PACK_REGULATORY_ISOLATION_TARGET_ARCHITECTURE_2026-07-26.md`
- 1 `docs/country-pack/COUNTRY_PACK_REGULATORY_ISOLATION_IMPLEMENTATION_2026-07-26.md`
- 1 `scripts/policy-gates-integration-contract.json`
- 1 `package.json`
- 1 CI and release workflows
- 1 Gate implementations under `scripts/`
- 1 Relevant code under `app/`, `actions/`, `services/`, `components/`, `hooks/`, `lib/`, `config/`, `prisma/`, and focused tests
- 1 Relevant dependency information under `graphify-out/`

Mission

From the source report and current repository evidence, isolate every material gate or constraint preventing Stoquify from progressing through smooth, sustained development.

For every blocker, provide a systematic resolution strategy covering:

- 1 Technical remediation
- 1 Architecture or data-model remediation
- 1 Human decision, review, or approval
- 1 Legal, accounting, regulatory, or external-expert dependency
- 1 Infrastructure, secrets, provider, or environment dependency
- 1 Product, operational, ownership, or coordination dependency
- 1 Verification and final exit criteria

Translate “make giant strides” into a measurable operating strategy that:

- 1 Restores a reliable everyday development path.
- 2 Restores a green integration and promotion path.
- 3 Separates release-only and external blockers from ordinary engineering work.
- 4 Preserves universal accounting, security, audit, tenant, privacy, reconciliation, and data-integrity controls.
- 5 Establishes one binding critical path rather than generating more disconnected audit reports.
- 6 Identifies the first product vertical slice that can advance without waiting for qualified statutory approval.

Required Review Board

Activate the full reviewer roster because this is a platform-wide blocker review. Require a finding or an explicit not applicable determination from every applicable discipline:

- 1 Enterprise/platform architecture
- 1 Backend, domain, API, event, and integration engineering
- 1 Database, data integrity, and migration engineering
- 1 Application security, IAM/RBAC, privacy, fraud, and abuse resistance
- 1 Frontend and design-system engineering
- 1 Workflow UX, accessibility, localization, and content design
- 1 Product strategy and business-process analysis
- 1 Finance, accounting, treasury, reconciliation, and internal controls
- 1 OHADA/SYSCOHADA statutory and country-pack compliance
- 1 Quality engineering and release assurance
- 1 SRE, DevSecOps, observability, resilience, performance, and cost
- 1 SaaS modularity, packaging, billing, growth, customer success, and product operations
- 1 Audit evidence, records governance, and data quality
- 1 POS, inventory, purchasing/AP, payroll, payments, close assurance, and offline synchronization
- 1 Change management, documentation, training, support, and rollout operations

Required Blocker Classification

Classify every discovered item into exactly one primary category:

Prevents ordinary local development, type checking, focused testing, migrations in safe environments, or feature integration.

1 Development blocker

Prevents code from entering the shared integration path but should not block unrelated local work.

1 Integration or promotion blocker

Must prevent production release but should not unnecessarily stop development.

1 Production-release blocker

Requires legal, accounting, statutory, provider, regulator, security, or other qualified approval that cannot be manufactured in code.

1 External or qualified-human blocker

Requires secrets, safe database targets, provider credentials, deployment infrastructure, or environment configuration.

1 Infrastructure or environment blocker

Results from unclear ownership, excessive work in progress, parallel workstreams, report sprawl, missing decision rights, or lack of a binding critical path.

1 Organizational or operating-model blocker

Requires monitoring or later remediation but does not currently block development or release.

1 Non-blocking risk or warning

Appears blocking only because a sequential command stops early, a missing alias hides downstream results, or a gate is assigned to the wrong lifecycle stage.

1 False or indirect blocker

Do not count the same root cause repeatedly merely because several aggregate commands expose it.

Required Analysis

1. Establish current repository truth

Inspect the dirty worktree and preserve all unrelated changes.

Reconstruct the complete gate topology from:

1 package.json

1 Gate scripts

1 Integration contracts

1 CI workflows

1 Release workflows

1 Migration and deployment scripts

1 Existing reports

Identify:

1 Every gate

1 Its owning script

1 Its lifecycle stage

1 Its dependencies

- 1 Whether it is wired into aggregate commands
- 1 Whether it is missing an expected alias
- 1 Whether it is fail-fast or continues downstream
- 1 Whether it mutates state
- 1 Whether its failure is direct, indirect, or environmental
- 1 Which gates become invisible when an earlier command fails

2. Record command evidence

Record each attempted command as:

- 1 passed
- 1 failed
- 1 skipped
- 1 timed out
- 1 blocked

For every command, record:

- 1 Exact command
- 1 Purpose
- 1 Start and completion time
- 1 Exit status
- 1 Concise result
- 1 Direct blocker
- 1 Whether the result is deterministic
- 1 Whether an environment or external dependency affected it

A failed gate is evidence to analyze, not permission to delete, weaken, bypass, or relabel it.

3. Separate OHADA rules from universal invariants

For each gate, determine whether it protects:

- 1 An OHADA/SYSCOHADA-specific statutory rule
- 1 A country-pack rule
- 1 Qualified expert approval or dated legal provenance
- 1 A universal accounting invariant
- 1 Ledger or monetary precision
- 1 Tenant isolation or RBAC
- 1 Audit or approval history
- 1 Privacy or secrets safety
- 1 Migration and data-loss safety
- 1 Idempotency or reconciliation
- 1 Provider reliability

1 General code quality or operational readiness

Explicitly identify which OHADA-specific activities can be deferred without weakening universal platform invariants.

4. Build the master blocker register

Create one row per unique root blocker with at least:

Blocker ID	Stable identifier such as BLK-001
Blocker	Concise name
Gate or command	Exact gate exposing it
Category	One classification from the required taxonomy
Lifecycle stage	Local, integration, promotion, pilot, or production
Current evidence	File, command, result, and date
Root cause	Underlying cause, not merely the failure message
What it actually blocks	Exact development or release capability
What it does not block	Work that can continue safely
Universal or OHADA-specific	Explicit classification
Technical remedy	Concrete engineering actions
Human remedy	Decision, review, approval, training, or ownership action
External remedy	Provider, regulator, expert, credential, or environment action
Dependencies	Preconditions and downstream effects
Responsible role	Role accountable for resolution
Decision authority	Role authorized to accept or approve the outcome
Verification	Exact test, command, evidence, or review
Exit criteria	Objective definition of resolved
Priority	Critical, high, medium, or low
Recommended phase	Immediate, next, later, or release-only
Status	Open, partially resolved, externally blocked, or resolved

5. Produce a resolution playbook for every blocker

For each material blocker, explain:

- 1 Why the blocker exists.
- 2 Whether it is correctly positioned.
- 3 Whether it is blocking too early or too broadly.
- 4 What must remain fail-closed.
- 5 The minimum safe technical remediation.
- 6 The required human or organizational action.
- 7 External dependencies that engineering cannot complete.

- 8 The correct owner and decision authority.
- 9 Dependency order.
- 10 Verification and rollback expectations.
- 11 What becomes unblocked when it is resolved.
- 12 What will remain blocked afterward.

Do not use “fix the gate” as a remediation. State the exact underlying condition that must be corrected.

6. Evaluate known blocker families

Validate rather than blindly repeat the report's findings. At minimum, investigate:

- 1 Missing or incomplete integration-policy profiles
- 1 Missing gate aliases or aggregate-command wiring
- 1 Production statutory gates incorrectly affecting development
- 1 Type-checking and code-health failures
- 1 Payroll immutability runtime validation
- 1 Prisma destructive-migration findings and approval evidence
- 1 Safe database-target requirements
- 1 Secrets, public URLs, HTTPS, and live provider configuration
- 1 Country-adapter official specifications and sandbox conformance
- 1 Release-isolation and pilot-evidence requirements
- 1 Gates with unexpected side effects
- 1 Report and readiness-artifact sprawl
- 1 Excessive parallel workstreams and lack of a single critical path
- 1 External approvals being mixed with engineering completion
- 1 Aggregate commands hiding downstream gate results

7. Design the execution sequence

Produce a dependency-aware plan with four horizons:

First 48 hours

Prioritize actions that restore accurate development feedback and remove false blockers, including:

- 1 Freeze a trustworthy evidence snapshot.
- 1 Reconcile current gate topology.
- 1 Restore or correct integration-only gate wiring.
- 1 Separate development, integration, and production commands.
- 1 Resolve type-checking uncertainty.
- 1 Confirm which failures are genuine versus environment-caused.
- 1 Assign named role ownership to every critical blocker.

First 7 days

Prioritize:

- 1 Local runtime and migration harness reliability
- 1 Payroll immutability verification
- 1 Migration-safety decision packets
- 1 Secrets and environment ownership
- 1 Safe database-target configuration
- 1 One green integration path
- 1 Selection of a releaseable vertical product slice
- 1 Elimination of duplicate blocker tracking

Next 30 days

Prioritize:

- 1 Completion of the selected vertical slice
- 1 Pilot and browser evidence
- 1 Release-isolation proof
- 1 Provider and country-adapter conformance
- 1 Qualified statutory review preparation
- 1 Operational documentation and support readiness
- 1 Sustainable CI and gate-governance ownership

Release-only track

Keep external statutory approval, production credentials, live provider conformance, and other legitimate production conditions visible without allowing them to stop unrelated product engineering.

8. Establish the operating model

Recommend a practical blocker-management cadence that includes:

- 1 One authoritative blocker register
- 1 One binding critical path
- 1 Explicit work-in-progress limits
- 1 A single owner and decision authority per blocker
- 1 Daily review of critical blockers
- 1 Weekly lifecycle-gate review
- 1 Clear escalation deadlines for human and external dependencies
- 1 No new broad audit unless it changes a decision
- 1 Every task must end in one of:
 - 1 Verified code or product evidence
 - 1 An explicit user or owner decision
 - 1 A precise external blocker with owner and next action
- 1 Superseded reports must be marked as historical rather than treated as current truth

Expected Artifacts

Create:

- 1 `what-next/stoquify-development-blocker-elimination-strategy.md`
- 2 `what-next/stoquify-development-blocker-register.json`

The Markdown report must contain:

- 1 Executive verdict
- 1 Current gate topology
- 1 Command-result ledger
- 1 Master blocker register
- 1 OHADA-specific versus universal-invariant matrix
- 1 Development versus integration versus production blocker matrix
- 1 Detailed resolution playbook
- 1 Critical-path dependency sequence
- 1 First-48-hours, first-7-days, next-30-days, and release-only plans
- 1 Ownership and escalation matrix
- 1 Recommended vertical slice
- 1 Operating cadence
- 1 Decisions required from leadership
- 1 Residual risks
- 1 Clear list of work that can proceed immediately

The JSON register must be machine-readable and contain the same unique blocker IDs and statuses as the Markdown register.

Verification Commands

Begin by discovering the actual commands currently defined in `package.json`. Do not invent missing commands or silently substitute aliases.

Run the relevant existing commands, including where available:

- 1 `git status --short`
- 1 `npm run typecheck`
- 1 `npm run policy:gates`
- 1 The integration-policy aggregate command
- 1 Focused statutory development and production gates
- 1 Focused inventory, AP, payroll, country-adaptor, and regulatory-boundary gates
- 1 `npm run workflow:assurance:runtime-check`
- 1 `npm run prisma:validate`
- 1 Existing migration-safety checks
- 1 Existing payroll immutability checks
- 1 Existing secrets and release-preflight checks

1 Existing local release-evidence checks

If a required alias or command does not exist, record it as `blocked` or `skipped` with evidence. Do not create it during this review.

Do not run destructive migrations, production deployments, live provider calls, or commands requiring unsafe credentials.

Risk Controls

- 1 Do not modify, remove, weaken, bypass, reorder, or suppress gates during this review.
- 1 Do not interpret statutory deferral as permission to weaken accounting, security, audit, tenant, privacy, reconciliation, or data-integrity controls.
- 1 Do not recommend placeholder financial models that would require destructive migrations later.
- 1 Preserve ledger precision, monetary semantics, provenance, idempotency, reconciliation, immutable evidence, and approval history.
- 1 Preserve fail-closed behavior at production boundaries.
- 1 Do not simulate or manufacture legal, accounting, security, or statutory approval.
- 1 Do not make legal, tax, accounting, security, privacy, accessibility, or production-readiness certification claims.
- 1 Do not expose secrets or production data.
- 1 Do not overwrite unrelated work.
- 1 Do not clean up unrelated lint findings.
- 1 Do not perform broad refactoring.
- 1 Do not convert production-release requirements into ordinary development requirements without justification.
- 1 Do not classify a warning as a blocker unless evidence shows what it actually prevents.
- 1 Do not solve organizational problems by adding more gates, reports, or process layers without measurable value.

Success Criteria

The review is complete only when:

- 1 Every material gate and unique root blocker has an evidence-backed classification.
- 1 Sequentially hidden downstream gates have been independently evaluated where safe.
- 1 Development, integration, promotion, pilot, production, external, and organizational blockers are clearly separated.
- 1 OHADA-specific requirements are separated from universal platform invariants.
- 1 Every blocker has a technical, human, external, or organizational resolution path as applicable.
- 1 Every blocker has an owner, decision authority, verification method, and objective exit criteria.
- 1 The analysis states exactly what each resolution would unblock.
- 1 Late statutory integration risk is evaluated across data, services, workflows, controls, UI, infrastructure, and operations.
- 1 A dependency-aware critical path is provided.
- 1 The first 48 hours, first 7 days, next 30 days, and release-only work are explicitly defined.
- 1 One vertical product slice is recommended for focused advancement.
- 1 The verdict is candid, prioritized, and actionable.

- 1 The team can identify what work may proceed immediately without weakening legitimate controls.

Non-Goals

- 1 Do not implement any remediation.
- 1 Do not disable or delete gates.
- 1 Do not integrate or remove the OHADA compliance layer.
- 1 Do not redesign unrelated modules.
- 1 Do not perform broad code cleanup.
- 1 Do not approve destructive migrations.
- 1 Do not configure production secrets or credentials.
- 1 Do not contact regulators, providers, or external experts.
- 1 Do not declare the system production-ready, legally compliant, secure, or accounting-certified.
- 1 Do not produce another general platform audit without a blocker-level decision and action register.